

Lec 63 Hands-on on LLM

Lec 63 Hands-on on LLM

Lecture	63 / 63 + intro
Duration	32:56
Captions	English (auto-generated)
Words	4,592
Video	V0q393vUcKw
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Good morning everyone. Uh this is Dwarpan. I am a fourth year PhD student in the electrical engineering department and uh working on reliable AI and LLMs. So I'll be presenting the tutorial on LLM hands-on today. This tutorial covers the next token prediction and causal self attention. We also implement a tiny decoder only transformer which is trained from scratch. We'll take a um toy example and show how these transformers are trained.

[00:36] We'll also talk about character based tokenization. We'll also shortly show small demo of bite pair encoding and wp piece tokenizations that are used in advanced models of large language models. We'll also try to load a small LLM distbook and uh generate using generate the response using greedy top K and nucleus or top decoding and we'll also show how basic prompt engineering works in this context. So whatever theories are covered in context to these um LLMs and uh you know training LLM uh we'll show some demo of how they are actually trained at scale. We'll show a toy and scale down version of it.

[01:32] Okay, first we need to install couple of uh dependencies um that are there um to go ahead with this tutorial uh including a suitable transformer version and

u accelerate and pandas. Once that is done then we need to um import couple of packages like math random um numpy pandas and different torch utilities right and then because uh to control uh uh the randomization and to ensure reproducibility you need to always set a seed right so we'll also write a function to set uh the seed to 42 and uh we need to check the device uh we should ideally be using a GPU which should show the device to be good.

[02:26] Now coming to next token prediction how it works for input tokens X_1 till X_T a causal language model predicts the next token at each position training minimizes the cross entropy between the predicted distribution and the actual next token. Okay. So and what is the difference between a decoder only model and the typical transformer? In a typical case there is an encoder and the decoder. However, what people found is that a decoder only large language model is sufficient because if it is trained at scale using large chunk of data and you know large model size then decoder itself is sufficient. Encoder is not required. In a decoding style model, what happens is that the model sees previous tokens and predicts the next

[03:14] token and it never sees the future token. Uh so uh so it uses a triangular causal mask while training because while training also it should not see the future tokens because in runtime that is in the inference time you it has to generate the tokens seeing the previous tokens only. Okay. Uh now let's define scaled dotproduct attention using query key value of q, v and causal equal to true. Um we we calculate d_k uh equal to q dot size minus one and we we define the scores as $q \cdot k$ pose divided by square root of d_k which you might have learned in the theory of attention.

[04:01] So this is a basic single head uh attention we are self attention we are implementing. Now if causal is true then you also calculate the triangular mask that we have talked about to ensure that during training the model learns to predict the next token just only uh based on looking the previous tokens and not the future tokens. Okay. So and coming to the weights um you get torch dossoft max of the scores and return the weights multiplied by the uh value uh vector that is the self attention and then qkv is defi you can try like random matrix torch rand and and and you can calculate the output and weights using the scaled dot product attention this is a quick check of the function that you have implemented um

[04:51] now um you can print those values. You can see that um you know uh you can see that the uh the upper diagonal values are zero which actually ensures that this is a triang upper triangular mask that is in place and which prevents the model during training phase to learn from the future tokens. Okay. Um now there are different tokenizers uh which need to be used while training LLMs because it cannot directly process the entire sentence. So it has to break the sentence into smaller pieces which are called tokens. Now the most simple tokenizer you can think of is like character based tokenizer which takes simply the different characters as different tokens. However, going forward, we will see that the advanced

[05:37] TLMs use some sophisticated tokenizers which breaks a word typically into two to three tokens which has meaningful uh which are like meaningful um subpieces of a word. Okay. Um okay. Now coming to our basic implementation where we use the character tokenizer. It is um used by the tiny model that we are going to train. So we take some training text. This is typically the corpus. So you can see we have taken a simple paragraph in our case where where in real case it will be like so often people talk about crawling the entire web or entire Wikipedia or you know entire corpus of documents that is available over the internet. So that is supposed to be this training test in case of a large model. Whereas we take

[06:26] this simple paragraph. Now uh this is too small. So we repeat we simply take this text and uh we we replicate this 80 times that becomes our entire corpus. Then um then you define characters as sorted of set of the corpus. So it simply takes the unique characters that are present and then then you define a dictionary for string to in integer conversion. So every character is assigned to an integer. uh uh so you need to fetch them. So you define string to uh integer um as character colon I and integer to string as um integer colon character. Okay. So so now you you can uh based on them you can write the encoding and decoding functions. So given some text what encode function does is um it says

[07:29] unknown equal to sorted of set of text minus set of so it checks what are the characters that it has never seen in the training purpose. Okay. So that it can't really process. So so so you simply raise an error that the characters are absent from the vocabulary. Otherwise if that is not the case, you only see the characters which you have encountered during the training. Then you simply um return the characters like integers uh uh you know indexes that represent the different characters that you are seeing in the text. Okay.

[08:02] So simply this is a encoding character encoding of the text that you are providing to the function encode. Okay. And then decode function if you give some ids like these indexes right uh and then that simply joins them after checking what that integer represents what character that integer represents. So you get the data so you you define sorry you define the data starch.tensor tensor of encode of the corpus and uh you calculate the you print all the things like what is the vocabulary size what is the training tokens and um and then you try like encoding the word transformer this string and then decoding it. So ideally if you decode encode and decode functions are proper then encoding something some text and then decoding it

[08:48] back should give you give the word back or give the string back that you can see that uh it encoded transformer to these integers which represent those characters in the vocabulary okay in the dictionary and uh if you decode it back this encoding function then it gives you back the transform. Okay. Uh so this was a quick uh sanity check that our functions are correctly implemented. Okay. Um now let's move to the uh next token batches. As we as we already discussed in the theory class and also at

the start of the tutorial these LLMs are trained on next token prediction based on the previous tokens. So you need to define uh the data set for the next token prediction and apply the loss on it and then train the model so that

[09:35] it becomes capable of uh predicting the next token. Okay. Um so we are going to define uh each target sequence as the input shifted left by one token. So that's the next token prediction task. Um so number of consecutive tokens used as one training sequence. Each input sample will contain 64 characters. So the block size is 64. Uh that's in LLM typically called context size context window. Okay. And number of independent sequences processed together during the training which is the batch size in the typical neural network trainings.

[10:23] And then we use the 90% of the complete token sequence for training. So you just create a 90% split and tokens from the beginning up to uh the split point from the training set. So you define train data and then rest 10% is the validation data. Then you define a function called get patch um which uh simply selects the train portion of the data. If if the partition is equal to if it is um anything else then you simply return the validation data.

[10:58] Right. Okay. Um so essentially it returns you the X transfer to device which is generally the GPU if it is available and the levels that is the next tokens to the device X and Y it returns. So X demo and Y demo is get batch train that X demo Y demo becomes the training split and uh you know you can print the shape of X demo and Y demo. Uh so what it turns out is that and then you can print a particular case uh of the example just to see. So input is um 32A 64 target is also 32A 64 and how this inputs and targets typically look like this is like this input is model compares its prediction with the correct next and then you shift it by one that becomes your target. So m is missing and you go next becomes next. So that is the

[12:02] next token prediction you need to do. Now uh let's try to go to the model. So we'll try to define a tiny decoder only transformer. The model has token and position embeddings and causal multi head self attention blocks fit forward layers residual connections layer normalization and a vocabulary prediction. The transformer encoder layer is used only as a reusable self attention block. Supplying a causal mask makes the complete network decoder only.

[12:33] Okay. So there is no encoder decoder cross attention in this case for the decoder only transformer. Um you define the configs vocabulary size block size um hidden dimension of the uh uh model number of heads and number of layers and dropout. Now you can define the uh decoder only transformer uh tiny decoder lm. You can inherit the nn domodule for this case and then you use the config and you know token embedding using the vocabulary size and d model.

[13:10] Then you create a learnable embedding for every position in the sequence uh self.position embedding and you can define the transformer block. After that that

leads to um uh that leads to the basic unit of the of the model and then you can simply repeat this uh by defining something called self.blocks is um the layer and then number of layers you want to repeat it and you need to finally use a normalization layer layer norm and convert this hidden vector into a logit per vocabulary token. So number of vocabul number of characters it had seen or number of tokens it had seen it can predict from that final norm stage. So you simply use a linear layer to predict from that final layer to the which vocabulary to

[13:58] be picked which characters to be picked given the previous tokens. Um yeah so so um so it's simply initialized and then uh and the forward pass simply involves um passing the uh you know converting the string to the embedding and passing the embedding to the language model using the mask. You can check this model definition on your own once more. And then finally here we define tiny model equal to tiny decoder lm config to the device.

[14:41] And then you can pass x demo and y demo uh through the model tiny model and you get the logits and initial loss prints the then you can print the parameters that are there and the logits and initial loss that you get is 3.613 613 and the you know the number of trainable and non-trainable parameters that the model has based on our definition are 413 and 987. Okay. Uh here we go. Uh we need to train this model. So for that uh we need to we we um we first uh you know write a function to estimate the loss uh you know model given the model and batches. So model eval and uh given uh in given the partition of train or val it computes the losses by passing the batches through the model and you can set the model to train um

[15:55] after the loss estimation is done and you also define the optimizer which is atom optimizer we use with the um initial alert setting of three um 10^{-4} and number of training steps is 400. If a device is CUDA else 180 because it will take a long time otherwise and you you uh repeat this training write the training loop and repeat it for number of training steps that you have defined in the config. Yeah. So if uh for every uh 50 step 50th step you just estimate the loss and print the loss part of it.

[16:36] Um and when the step equal to equal to training steps you break out of it. After uh during the training after loss is calculated you need to as generally forum torchbased neural network training you need to set uh optimizer.0 grad and loss.backward which will update the model parameters using gradient descent. Yeah. So and optimizer.step step and uh you simply print the training time that so if you run this in everything is run in collab only so you can try this notebook as it is um I'll make sure that this collab notebook is made available um so you can see that the train loss starts with 3.623 623 which was the initial loss and then it keeps uh you know uh keeps lowering keeps going down with steps and finally it reaches.161

[17:33] and validation loss you should also track to ensure that the training loss is

not reducing because of simply overfitting. So you can see that the val validation loss also keeps going down in equal proportion. Uh here we go. Our model is trained and we need we we can now uh probably it was covered in the theory part that you typically encounter three types of decoding modes which are most commonly used in case of LLMs which are greedy decoding, top K decoding and nucleus decoding or top B decoding. In greedy decoding you need to choose the maximum probability token. So given previous tokens the LM gives a gives assigns the probability to all the tokens that are present in the vocabulary. So you can

[18:20] pick the maximum probability token that is called the greedy um decoding. So it has no stoasticity. No matter what um uh you know what whatever K or you know whatever stoastic sampling or temperature you use you'll only get the same um output again and again. If you use the gritty decoding mode. Now if you use top k decoding in stoastic mode or you know um temperature more than zero or like typically like temperature equal to one then it will retain only the top k largest logits that are assigned to the vocabulary and sample from that set.

[18:58] Whereas the other way around is nuclear nuclear sampling or top p sampling. In this case you can define p equal to say 0.9. So it will only use the vocabulary top vocab top tokens whose logits or you know probability is sum to greater than equal to 0.9 and from that set it will sample that is called nucleus or top p sampling. We'll try uh you know implement all of them. So apply top k top k is minimum of um top k and logits dot size like the vocabulary size and then the cutoff is um is based on the top key and you just return um logits dot masked field logits less than cutoff and apply top p is based on as I said based on the pro top probability that you define um based on the cumul relative probability

[19:56] from the vocabul and here we write a generate function for the tiny language model we have built. Uh generate tiny you need to provide arguments like the model prompt decoding strategy you want to use max new tokens you want to use like how many maximum tokens it can generate while generating the response. it can't really uh keep generating infinite number of tokens, right? And u the temperature you need to set uh as you might know the temperature equal to zero ensures that there is no stoasticity in the sampling process. So uh and you keep increasing the temperature and it introduces more and more stoasticity by going towards the uniform distribution of the um tokens in the vocabulary.

[20:48] Yeah. So uh you in you you set the temperature to 0.9 and then top k as 10 and top key as 0.9. Okay. And you put the you can put the model to the eval mode uh to ensure that there is no gradient flow in it while doing the inferences. And um and here how this is how you need to create the uh uh you know generation process. based on the different decoding study. U this is auto regressive process. You can go through this implementation again based on what theory you have learned is it's simply u implementation of that.

[21:36] Okay. And uh and here is an example in the greedy mode. It generates a transformer processes tokens using self attention and feed forward layers. causal attention prevents a token from reading tokens in the future during training the uh you know based on the maximum token this got truncated and toply top sampling uh you can see that uh initially it's remains same but um while going uh forward it suddenly breaks the forward to um breaks this word this is because this um full stop was sampled from the top case set instead of the most probable token R, right? And uh and after that there is there are other stoasticities that are involved which makes the response little different from the greedy response. And

[22:32] coming to nucleus response which is another way of doing the uh stoastic sampling other than top K. Here also you can see that um similar uh you know um the responses are different in this second sentence. Greedy decoding chooses the probable next to um so this tiny model is intentionally weak. You can see the sentence formation and word formations itself are not are not that great. So it has a tiny corpus character tokens few parameters and only a few hundred updates. Its purpose is to just show uh how this mechanism work and training and inference and these decoding schemes how they work and uh and not to produce the high quality uh responses.

[23:17] Uh as we discussed um the character tokenization is a very basic tokenization because different words might use same characters but uh they mean their meanings may be so different from each other. So the large so the advanced language models use advanced tokenization schemes. Some examples are like wordpiece bite pair encoding or bp. They break the word into subwords which has meaningful uh meaningful um construction based on a combination of few words. Typically a word can be thought of as it broken down to two to three tokens. uh using these in tokenizers uh the transformers library in hugging face use the auto tokenizer. So these tokenizers are available in it. You define BP tokenizer and wp tokenizer

[24:13] from this um dgbd2 and b and given a set of strings transformer tokenization unbelievable and so on. uh you just can for your visualization and understanding how these uh tokenizers are different from their character tokenization you can print them. So you can see in these examples the transformer using bite pair encoding um breaks down to the tokens trans and former and the word piece tokenizer becomes transform and tokenization is broken into tokenization in bite where BP and it is broken down to token and then so here the tokenizations remain same by both tokenizers. Then unbelievable becomes four tokens in BP unbel ifable.

[25:13] Whereas wordpiece breaks it down into um just one token of unbelievable. A prompt engineering is broken down to prom, PT and ginger engineering and the word piece breaks down to prompt and engineering. Uh these are advanced tokenizers which are actually used in training this advanced um large language models which stays beyond our uh ability and purpose also to provide the training. And based on this you can um take a big project in front on yourself and then try to train a um little

larger LLM on using such encoding schemes and larger corpus.

[26:01] Um moving forward uh we'll also try to show a demo of a small LLM which is dist GPD2. Um it is a pre-trained base completion model not instruction tuned chat assistance. uh what what is covered in the course I think is that the LLMs typically go through two stage of training first they go through the pre-training on the large corpus and next they go through the instruction tuning to make the responses sound like humanlike okay so so we take the base which have not which has not gone through the instruction tuned version uh you can load uh from transformers library um the model and then um so you can see how small our parameters were when we trained our tiny ML. It has some 81 cres of parameters.

[27:03] So billions of parameters, right? several billion uh parameters. Yeah. Billionord ordered model billion parameter order model. Um and then in the in the uh you can simply write the um generate a response function like we have used for the model we had and uh you can set all the arguments like we did earlier and then you can check um the uh responses that are generated. So here is a set of responses generated using the dist GPD model um using greedy top key and nuclear sampling. So what you can see is that um the greedy top key and nuclear sampling vary from each other. So top key and nuclear sampling why they're important they often use stochasticity and sometimes the responses may be more meaningful because it it samples from

[28:16] the space of all probable different answers rather than sticking to a very um fixed hard response that is obtained by greedy sampling and here we inspect the effect of random seats. So if you um like we discussed and greedy doesn't depend on the seed. So here is an interesting demo of that. If you change the seed 1 2 3 using uh greedy decoding mode the answer doesn't change at all because it's always picking up the largest probability token while doing the decoding process.

[28:57] Whereas top k is stoastic inherently by its nature. So if you change the seat the response says keep bearing. And for nuclear sampling which is also stoastic the responses keep bearing if you change the seat. And uh here is a final demo of prompt engineering how it works. So how what is prompt? Prompt is simply like the models can generate free flow responses right but prompt is something that you you condition it. So you essentially provide a context of it and then it completes the sentence like transformers are you can feed this to the LLM and then let it complete. In a beginner machine learning textbook transformers are described as so like u three advantages of transformer models are like that. And if

[29:49] you um let the sentence be completed using distilled GPD2, it completes the completes those sentences like transformers are very very scary, very scary, very dangerous. So on so not so great um completions but this is how it works um mechanistically. Yeah. Um so the so here are few other exercises that you can change the D

model number of layers and number of heads um in the tiny LM training that you had done. Um you can also change other hyperparameters like block size, temperature and uh top ktop values and then experiment with uh larger corpus also and different tokenization schemes also and then uh you can see how your um responses get better and your how your trainings get better um with increasing

[30:45] value of these parameters. Um so here are a couple of exercises I've added in case you want to explore more. Um coming to the summary the key takeaways are causal language models learn next to next token prediction. Causal masks prevent access to future tokens. Tokenization is model specific. Yeah. And uh like you cannot use a different tokenizer during the response generation at inference time and different tokenizer for the training the same LLM they has to be same greedy decoding favors the determinism. Random seed doesn't affect it at all. Top K and nucleus sampling introduce controlled diversity. You can't you you don't want it to be completely random but you can use controlled stochasticity to get

[31:35] meaningful answers sometimes and the prompt wording can change the model's continuous distribution. So prompt is basically an incomplete context that is given and the LLM starts use it as a prior and then starts the generation process. So it uses this prompt as a context and um here are couple of official references you can go through to learn more about this amazing open-source projects and contributions that people have made which also helped me uh prepare this tutorial and works I think I guess all of us to understand and work on such um um great developments of large language models.

[32:22] I think for today that that is all u that I have for this tutorial. Um I'll make sure to uh make this notebook available for you to try out other things and use it as a base. Um I hope it was good to understand uh how the large language model building and then inferencing works uh given the theory that was covered part of the as part of the course. Um thank you very much for today.